CMSC 420: Extended KD Trees

Justin Wyss-Gallifent

January 19, 2026

1 Problems **2**

1 Problems

1. In an extended KD-tree when we split according to the largest spread we use the median of the node's data. What is a good reason not to use the mean?

   Solution:
   The mean isn't guaranteed to split in a balanced manner. For example in a KD-tree with $k = 2$ if the points were $(1, 1), (2, 1), (3, 1), (4, 1), (90, 1)$ we would split by the x-coordinate with a mean of $(1 + 2 + 3 + 4 + 90)/5 = 20$ which would put four points on the left and just one on the right.

2. Consider the following extended kd-tree. The tree is on the left and the plane diagram is on the right. The shaded boxes are the bounding boxes for the leaf nodes.

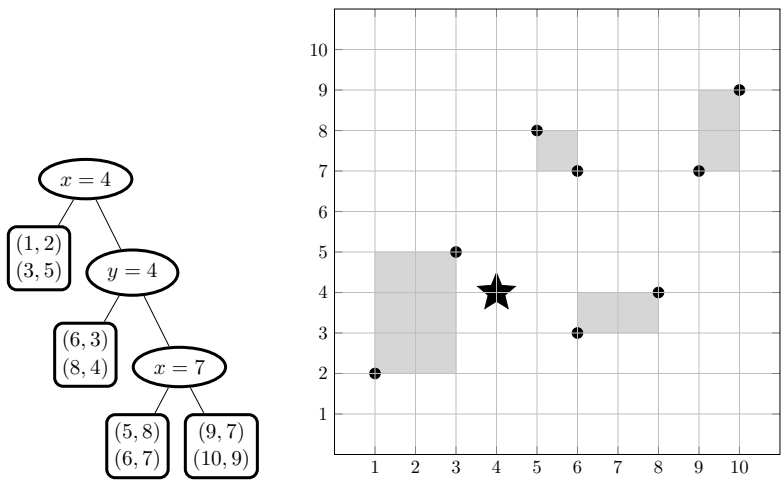

Figure 1: An EKD Tree and Some Bounding Boxes

(a) Draw separate plots of the bounding boxes for each non-leaf node.
Solution:
In order $x = 4$, $y = 4$, and $x = 7$ are:

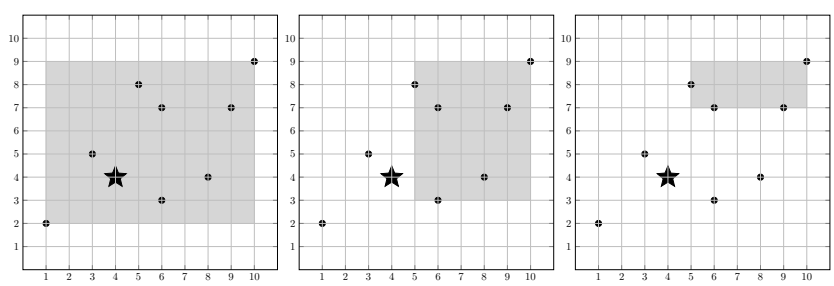

Figure 2: Bounding Boxes for the Non-Leaf Nodes

(b) Suppose we are looking for the two nearest neighbors to $(4, 4)$. We are following the algorithm discussed in class and used on the coding project. Walk through this algorithm, explaining each step briefly, and write down what the list looks like after each change

Solution:

The list is $L = []$. At the node $x = 4$. The left and right bounding boxes are equidistant so we look left first.

At that leaf node. We include both points so $L = [(3, 5), (1, 2)]$.

The list is $L = [(3, 5), (1, 2)]$. The list is full but the right bounding box is closer than $(1, 2)$.

At the node $y = 4$. The left bounding box is closer so we look there first.

At that leaf node. The point $(6, 3)$ is closer than $(1, 2)$ so we replace. Now $L = [(3, 5), (6, 3)]$.

The list is $L = [(3, 5), (6, 3)]$. The list is full and the right bounding box is further than $(6, 3)$ so we don't need to look there.

Thus we have $L = [(3, 5), (6, 3)]$.

3. Give a graphical example in 2D of a point P and two bounding boxes B_1 and B_2 such that all of the following are satisfied:

   - P is not in either bounding box.
   - Each bounding box contains exactly two points.
   - P is closer to B_1 than it is to B_2.
   - P is closer to both points in B_2 than it is to both points in B_1.

   Solution:
   Something like this, where B_1 is the large box and B_2 is the small box:

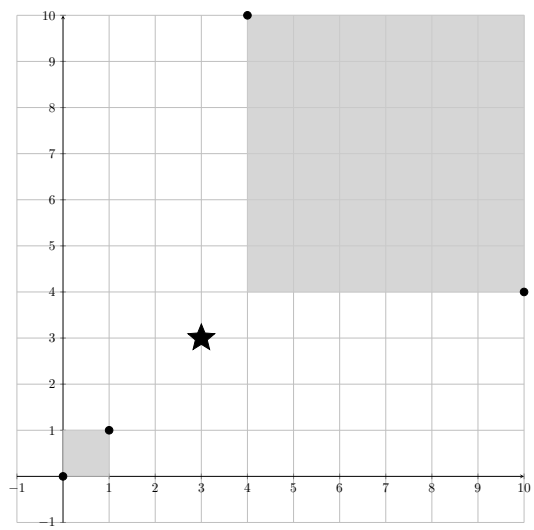

Figure 3: Solution to Problem

4. Consider the following extended kd-tree. The tree is on the left and the plane diagram is on the right. The shaded boxes on the right are the bounding boxes for each leaf. The bounding boxes for the (non-leaf) subtrees are not shown.

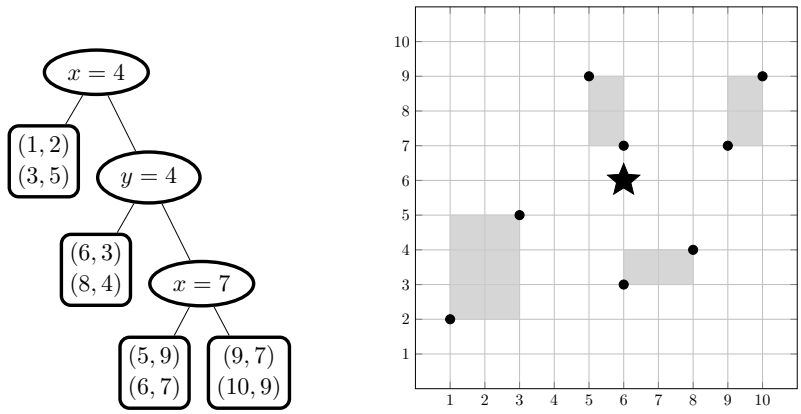

Figure 4: An EKD Tree and Some Bounding Boxes

Suppose we are looking for the three nearest neighbors to $(6, 6)$.

We are following the algorithm discussed in class and used on the coding project. For each of the following points determine whether the point will ever be on our list (even if it's removed later) and whether it will be on our list when the algorithm completes. Assume that when leaf lists are checked they are checked in the order the points are given in the tree (top-down). Fill in each space with either **Y** or **N**.

Point	**Ever in List?**	**In List at End?**
$(1, 2)$	N	N
$(3, 5)$	N	N
$(6, 3)$	Y	Y
$(8, 4)$	Y	Y
$(6, 7)$	Y	Y
$(5, 9)$	Y	N
$(9, 7)$	Y	N
$(10, 9)$	N	N

5. Assuming an EKD-tree with $k = 2$ and $m = 1$ using cycle splitting, give an example of a sequence of four insertions followed by one deletions which results in the tree having exactly two splitting nodes both of which split by x. Draw sufficient pictures to justify your process and result.

   Solution:
   First insert $(5, 5)$ and $(15, 5)$ which splits at $x = 10$:

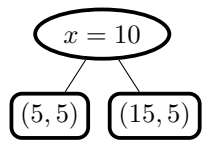

Figure 5: Inserting (5,5) and (15,5) Causes a Split

   Then we insert (20,15) which goes right and then splits at $y = 10$:

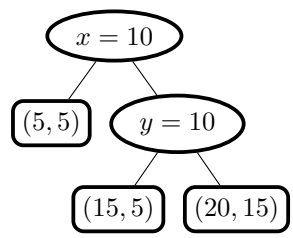

Figure 6: Inserting (20,15) Goes Right and Causes a Split

Then we insert $(40, 20)$ which goes right and then right and then splits at $x = 30$:

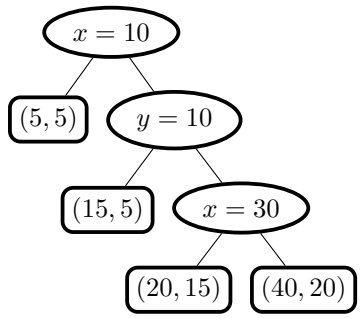

Figure 7: Inserting (40,20) Goes Right and Right and Causes a Split

Then we delete $(15, 5)$ which removes the $y = 10$ splitting node:

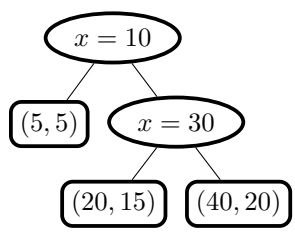

Figure 8: Delete (15,5) Which Removes the Splitting Node

6. Consider the following extended kd-tree. The tree is on the left and the plane diagram is on the right. The shaded boxes on the right are the bounding boxes for each leaf. The bounding boxes for the (non-leaf) subtrees are not shown.

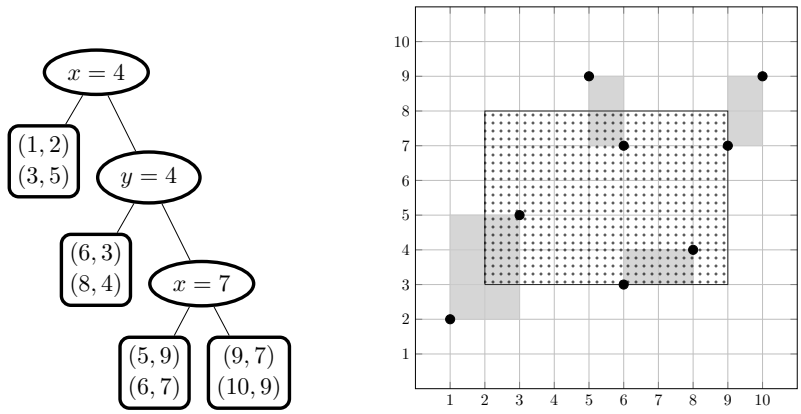

Figure 9: An EKD Tree and Some Bounding Boxes

Suppose we are performing a range query for points in the query box:

$$Q = [2, 9] \times [3, 8]$$

We are following the algorithm discussed in class except for a slight modification (also discussed in class) where - if we need to visit both subtrees of a splitting node we should first visit the subtree whose bounding box has larger overlap with Q, then the other one.

(a) In what order would the leaves be visited? Only consider those which would actually be visited.

First the leaf containing $(6, 7)$.
Then the leaf containing $(9, 7)$.
Then the leaf containing $(6, 3)$.
Then the leaf containing $(1, 2)$.

(b) What points would be in the list at the end?

$(3, 5)$, $(6, 3)$, $(8, 4)$, $(6, 7)$ and $(9, 7)$.

7. Consider the following extended kd-tree. The tree is on the left and the plane diagram is on the right. The shaded boxes on the right are the bounding boxes for each leaf. The bounding boxes for the (non-leaf) subtrees are not shown.

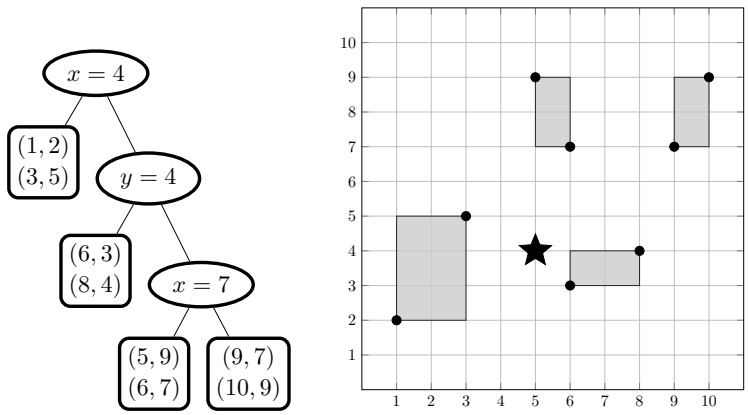

Figure 10: An EKD Tree and Some Bounding Boxes

Suppose we are looking for the three nearest neighbors to $(5, 4)$.

We are following the algorithm discussed in class and used on the coding project. For each of the following points determine whether the point will ever be on our list (even if it's removed later) and whether it will be on our list when the algorithm completes. Assume that when leaf lists are checked they are checked in the order the points are given in the tree (top-down). Fill in each space with either **Y** or **N**.

Note: Please do not use any other symbols, and empty boxes will be marked as incorrect.

Point	**Ever in List?**	**In List at End?**
$(1, 2)$	N	N
$(3, 5)$	Y	Y
$(6, 3)$	Y	Y
$(8, 4)$	Y	Y
$(6, 7)$	Y	N
$(5, 9)$	Y/N	N
$(9, 7)$	N	N
$(10, 9)$	N	N

We start at $x = 4$. The right BB is closer so we go right. Now we are at $y = 4$. The left BB is closer so we go left and $L = [(6, 3), (8, 4)]$. Then we back up to $y = 4$ and we go right to $x = 7$. The left BB is closer so we go left. In this leaf $(6, 7)$ is added to the list and $L = [(6, 3), (8, 4), (6, 7)]$. Then we back up to $x = 4$ and we go left. In this leaf $(3, 5)$ replaces $(6, 7)$ and $L = [(6, 3), (8, 4), (3, 5)]$. NOTE: At the BB containing $(5, 9)$ and $(6, 7)$ technically since the leaves are checked in top-down order we would add $(5, 9)$ to the list and then immediately replace it by $(6, 7)$. Thus we accepted either Y or N for $(5, 9)$ ever in list.

8. Consider the following extended kd-tree. The tree is on the left and the plane diagram is on the right. The shaded boxes on the right are the bounding boxes for each leaf. The bounding boxes for the (non-leaf) subtrees are not shown. Notice also that the leaf nodes have labels A, B, C, D.

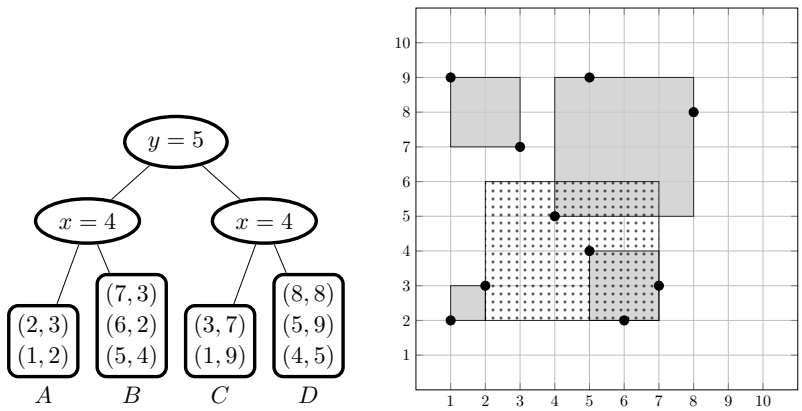

Figure 11: An EKD Tree and Some Bounding Boxes

Suppose we are performing a range query for points in the query box:

$$Q = [x_{min}, x_{max}] \times [y_{min}, y_{max}] = [2, 7] \times [2, 6]$$

We are following the algorithm discussed in class except for a slight modification (also discussed in class) where - if we need to visit both subtrees of a splitting node we should first visit the subtree whose bounding box has larger overlap with Q, then the other one.

(a) In which order are the leaf nodes visited?

Order	B, A, D

(b) In the following table identify the points which will be in our list. Fill in each space with either **Y** or **N**.

Point	**In List?**
$(2, 3)$	Y
$(1, 2)$	N
$(7, 3)$	Y
$(6, 2)$	Y
$(5, 4)$	Y

Point	**In List??**
$(3, 7)$	N
$(1, 9)$	N
$(8, 8)$	N
$(5, 9)$	N
$(4, 5)$	Y

9. In an EKD tree with $m = 4$ we have an overfull node containing these points:

$$(2, 1, 1), (5, 2, 6), (0, 4, 10), (1, 3, 0), (1, 1, 1)$$

If we perform a spread split on the points, fill in the splitting node data and leaf points below. The order of the points in the leaves doesn't matter.

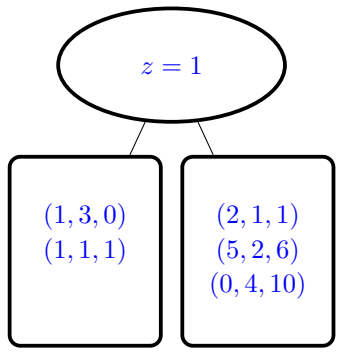

Figure 12: Spread Split Result

10. Given the following set of points:

$$(1,2,3,4),(3,1,2,4),(2,1,3,4),(4,1,3,2),(5,1,3,2),(5,2,3,5)$$

(a) Cycle sort them by the first coordinate.
The result is:

$$(1,2,3,4),(2,1,3,4),(3,1,2,4),(4,1,3,2),(5,1,3,2),(5,2,3,5)$$

(b) Cycle sort them by the second coordinate.
The result is:

$$(3,1,2,4),(4,1,3,2),(5,1,3,2),(2,1,3,4),(1,2,3,4),(5,2,3,5)$$

(c) Cycle sort them by the third coordinate.
The result is:

$$(3,1,2,4),(4,1,3,2),(5,1,3,2),(1,2,3,4),(2,1,3,4),(5,2,3,5)$$

(d) Cycle sort them by the fourth coordinate.
The result is:

$$(4,1,3,2),(5,1,3,2),(1,2,3,4),(2,1,3,4),(3,1,2,4),(5,2,3,5)$$

11. Suppose we start with an empty EKD tree with $m = 2$ and use cycle splitting.

    (a) Show the final result after inserting the points $(1, 3, 2)$, $(1, 2, 3)$, and $(5, 2, 3)$.

    The root has all three points before splitting and since we are cycle splitting we cycle sort by x yielding $(1, 2, 3), (1, 3, 2), (5, 2, 3)$ which has a median $x = 1$ and which divides up into the first one and the second two resulting in:

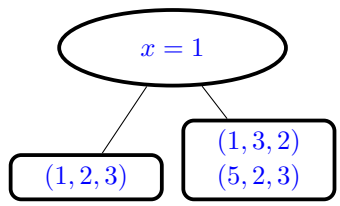

Figure 13: Cycle Split Result

    (b) Continuing from the above, show the final result after inserting the points $(0, 1, 1)$ and $(2, 5, 0)$.

    The point $(0, 1, 1)$ goes left which is not an issue and the point $(2, 5, 0)$ goes right which has $(1, 3, 2), (5, 2, 3), (2, 5, 0)$. Since we are cycle splitting we sort by y yielding $(5, 2, 3), (1, 3, 2), (2, 5, 0)$ which has a median $y = 3$ and which divides up into the first one and the second two resulting in:

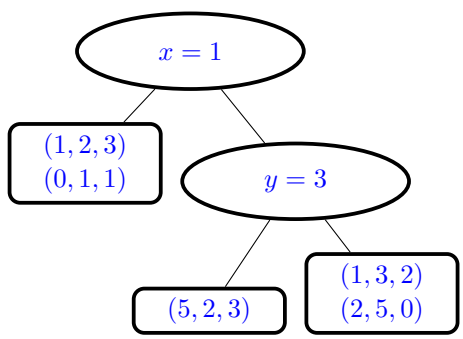

Figure 14: Cycle Split Result

12. Suppose we start with an empty EKD tree with $m = 2$ and use spread splitting.

    (a) Show the final result after inserting the points $(0, 2, 7)$, $(3, 1, 1)$, and $(1, 1, 2)$.

    The root has all three points before splitting and since we are spread splitting and the largest spread is z we cycle sort by z yielding $(3, 1, 1), (1, 1, 2), (0, 2, 7)$ which has a median $z = 2$ and which divides up into the first one and the second two resulting in:

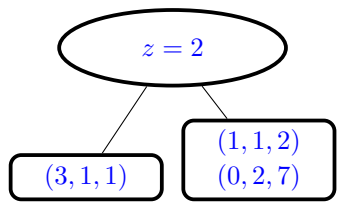

Figure 15: Spread Split Result

    (b) Continuing from the above, show the final result after inserting the points $(9, 1, 0)$ and $(0, 0, 1)$.

    Both of these points go left which has $(3, 1, 1), (9, 1, 0), (0, 0, 1)$ and since we are spread splitting the largest spread is x we cycle sort by x yielding $(0, 0, 1), (3, 1, 1), (9, 1, 0)$ which has a median $x = 3$ and which divides up into the first one and the second two resulting in:

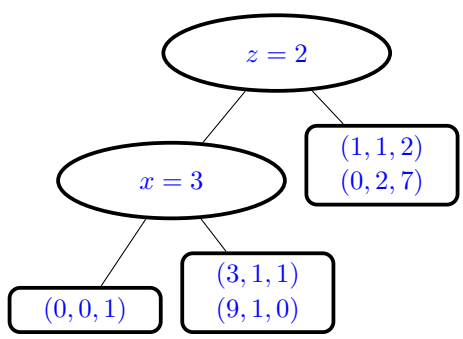

Figure 16: Spread Split Result